CSS - Display Grid

Profa. Ariadne Arrais

Por que aprender Grid?

- Grid resolve problemas de layout bidimensional;
- Permite organizar elementos em linhas e colunas;
- Ideal para:
  - Layouts de páginas completas;
  - Interfaces modernas.
- Mais poderoso que float e mais estruturado que flexbox.

Display grid

- É um sistema de layout que organiza elementos em linhas e colunas ao mesmo tempo;
- Controla linhas e colunas simultaneamente;
- Permite o posicionamento preciso dos elementos;
- Diferente do Flexbox:
  - Flexbox: apenas 1 dimensão;
  - Grid: 2 dimensões.

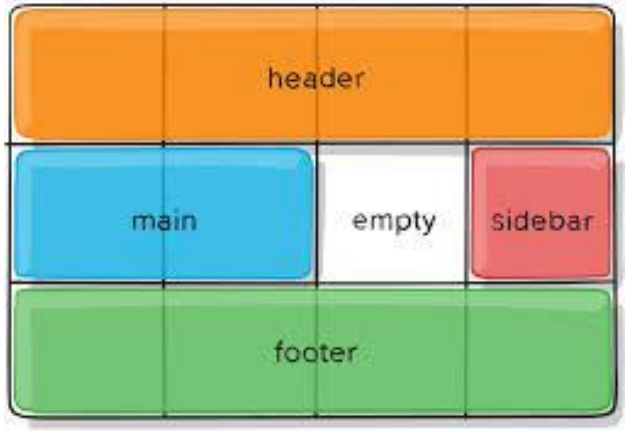
Definindo display do tipo grid

```
display: grid;
display: inline-grid;
```

Grid x Layout “padrão”

- Layout padrão:
  - Baseado em **block** e **inline**.

- Grid:
  - Baseado em **linhas** e **colunas**.



Estrutura do Grid

- Existem dois papéis diferentes:
  - Container (elemento pai)
  - Itens (elementos filhos)

Conceitos principais

- Grid lines
- Grid columns
- Grid rows
- Grid cells
- Grid areas

PROPRIEDADES DO ELEMENTO-PAI

Propriedades do elemento-pai

- Propriedades:
  - display;
  - grid-template-columns;
  - grid-template-rows;
  - gap;
  - grid-auto-rows / grid-auto-columns;
  - justify-self;
  - align-self;
  - place-self.

grid-template-columns

- Especifica o número (e a largura) de colunas em layout do tipo grid.

- Os valores são uma lista separada por espaços, onde cada valor especifica o tamanho da respectiva coluna

```
.grid-container {
        display: grid;
        grid-template-columns: valor1 valor2 valor3;
}
```

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-template-columns

grid-template-columns

- **none**: padrão (colunas automáticas)
- **auto**: tamanho baseado no conteúdo/container
- **max-content**: tamanho do maior conteúdo
- **min-content**: tamanho do menor conteúdo
- **length (px, rem...)**: tamanho fixo
- **% (percentual)**: relativo ao container
- **minmax(min, max)**: intervalo de tamanho
- **fit-content()**: ajusta sem ultrapassar o conteúdo
- **repeat()**: repete colunas - **repeat**(12, 1fr);
- **subgrid**: herda grid do pai
- **initial:** valor padrão
- **inherit**: herda do pai

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-template-columns

Unidade **fr**

- fr: fração do espaço disponível.
- Divide o espaço restante do container.
- Exemplo:
  - grid-template-columns: 1fr 1fr;
    - Duas colunas de mesmo tamanho.

  - grid-template-columns: 1fr 2fr;
    - Divide em três partes.
    - 1ª colunas – 1 parte; 2ª coluna – 2 partes.
- Considera apenas o espaço livre, primeira respeita os valores fixos (px ou %).

grid-template-rows

- Especifica o número (e a altura) das linhas em um layout do tipo grid.

- Os valores são uma lista separada por espaços, onde cada valor especifica a altura da respectiva linha.

- Exemplo:
  - grid-template-rows: auto auto;
  - grid-template-rows: 50px 100px;
  - grid-template-rows: 20% 20%;
  - grid-template-rows: 70px auto;
  - grid-template-rows: repeat(2, 60px 100px);

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_grid-template-rows

gap

- Define o tamanho do espaçamento entre as linhas e entre as colunas em layouts flexbox, grid ou multicolunas.

- É a abreviação simples das propriedades:
  - row-gap
  - column-gap

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_gap

grid-auto-rows / grid-auto-columns

- Define o tamanho das linhas em um container do tipo grid. Esta propriedade afeta apenas as linhas cujo tamanho não está definido.

justify-self

- Alinha toda a grade ao longo do eixo horizontal dentro de seu contêiner, desde que haja espaço extra não consumido pelas próprias faixas da grade.

- Valores:
  - start; end; center;
  - space-between; space-around; space-evenly;
  - stretch - estica as colunas do grid (especificamente as faixas com tamanho automático) para ocupar todo o espaço restante no container.

align-self

- Especifica o alinhamento para itens dentro de um container do tipo grid ou flexbox.

- Os itens do grid são alinhados na direção do bloco (de cima para baixo e da direita para esquerda).

- Valores: strech, start, end, center, baseline.

place-self

- Alinhas itens individuais do grid e é uma abreviação para as seguintes propriedades:
  - align-self
  - justify-self

https://www.w3schools.com/cssref/playdemo.php?filename=playcss_place-self

PROPRIEDADES DOS ELEMENTOS-FILHOS

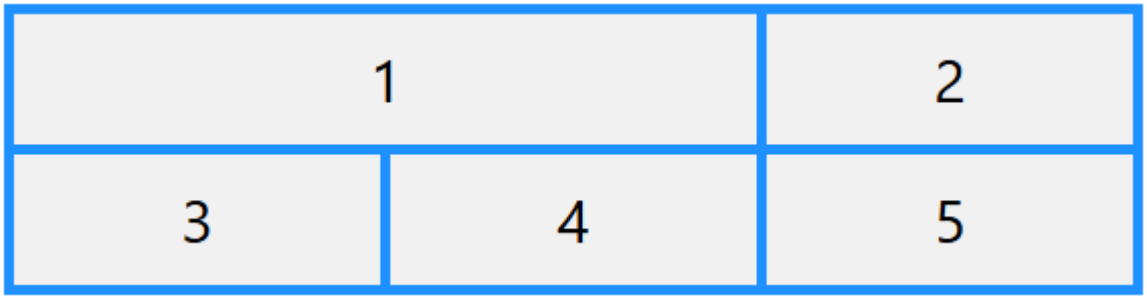
Propriedades

- **grid-column-start**: Especifica em qual coluna o item do grid começará.
- **grid-column-end:** Especifica em qual coluna o item do grid terminará.
- **grid-column:** Atalho para as propriedade *grid-column-start* e *grid-column-end.*
- **grid-row-start:** Especifica em qual linha o item do grid começará.
- **grid-row-end:** Especifica em qual linha o item do grid terminará.
- **grid-row:** Atalho para as propriedade *grid-row-start* e *grid-row-end.*

grid-column

- Define colunas ocupadas

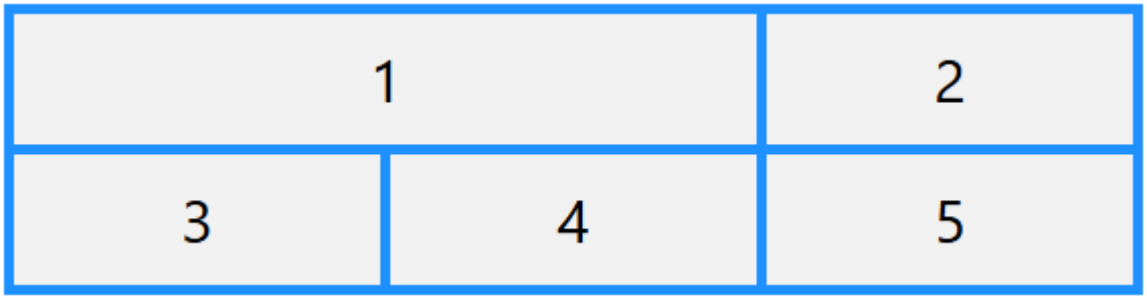
```
.item1 {
  grid-column-start: 1;
  grid-column-end: 3;
}
```



grid-column

- Define colunas ocupadas

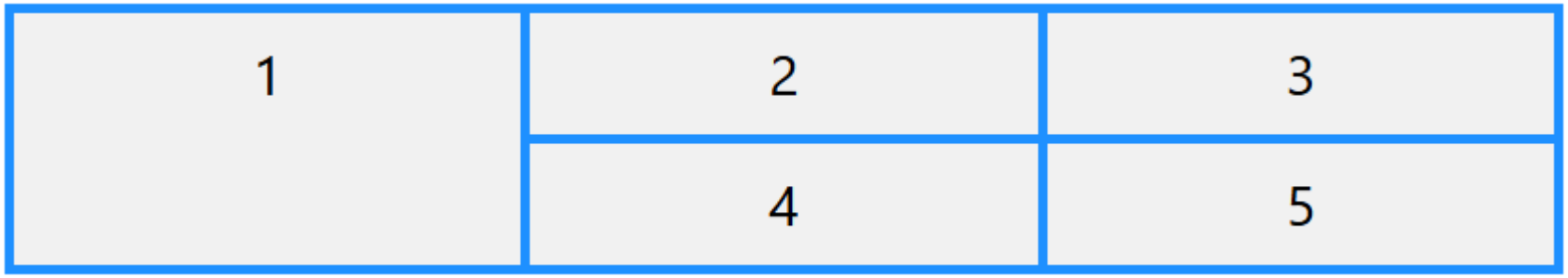
```
.item1 {
  grid-column: 1 / span 2;
}
```



grid-row

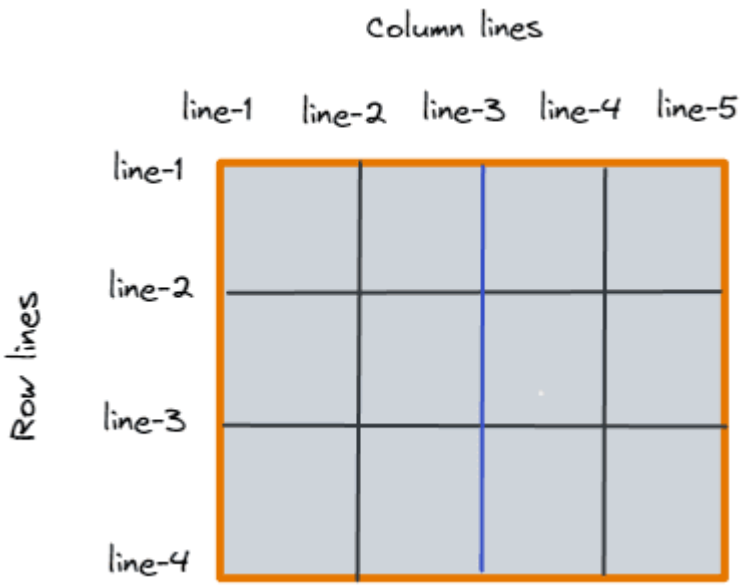
- Define as linhas ocupadas

```
.item1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
```



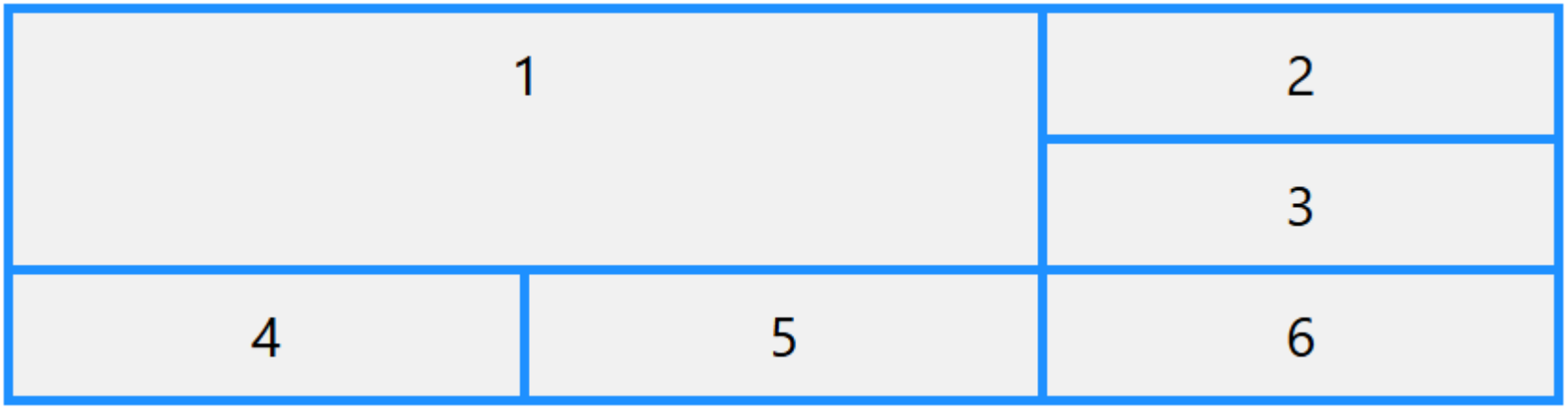
Combinando grid-column e grid-row

- Define linhas ocupadas
  - grid-row: span 2;



grid-area

```
.item1 {
  grid-column: 1 / span 2;
  grid-row: 1 / span 2;
}
```



grid-area

- Atalho para as propriedade *grid-row-start, grid-column-start*, *grid-row-end* e the *grid-column-end*

- grid-area: *grid-row-start / grid-column-start / grid-row-end / grid-column-end* | *itemname*;

https://www.w3schools.com/cssref/tryit.php?filename=trycss_grid-area

GRID TEMPLATES AREAS

Grid templates areas

- Responsável por definir o layout visual.
- Uma propriedade que permite definir uma estrutura de layout nomeada diretamente em CSS, atribuindo nomes às áreas da grade.
- Cada sequência de caracteres representa uma linha, e cada palavra nessa sequência corresponde a uma coluna, possibilitando layouts intuitivos, visuais e fáceis de manter, através da repetição de nomes para abranger as células.

Grid templates areas

```
grid-template-columns: 200px 1fr;
grid-template-rows: auto 1fr auto;
grid-template-areas: "header header"
                     "sidebar main"
                     "footer footer";
```

Prática

- https://cssgridgarden.com/

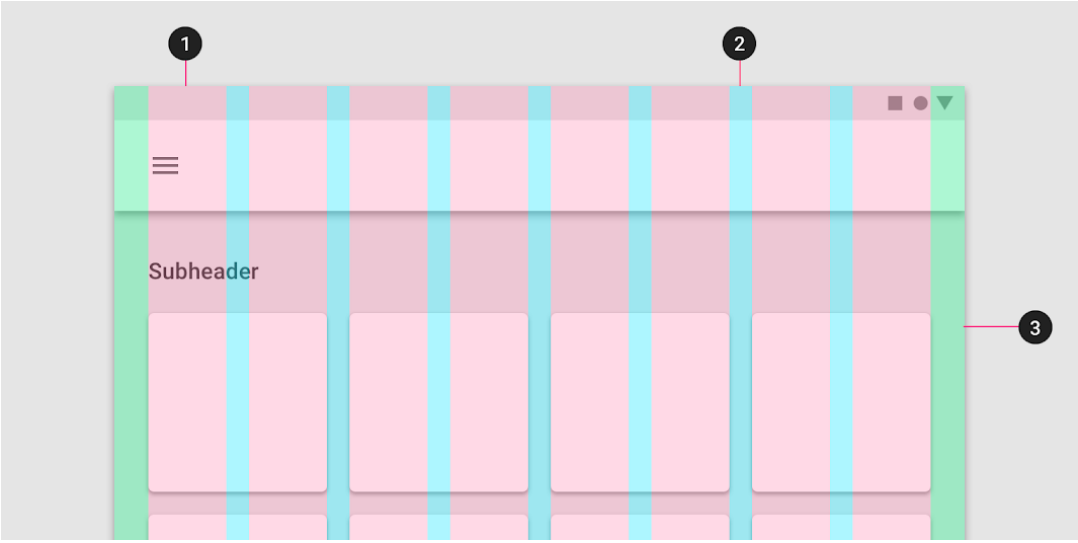
SISTEMA DE 12 COLUNAS

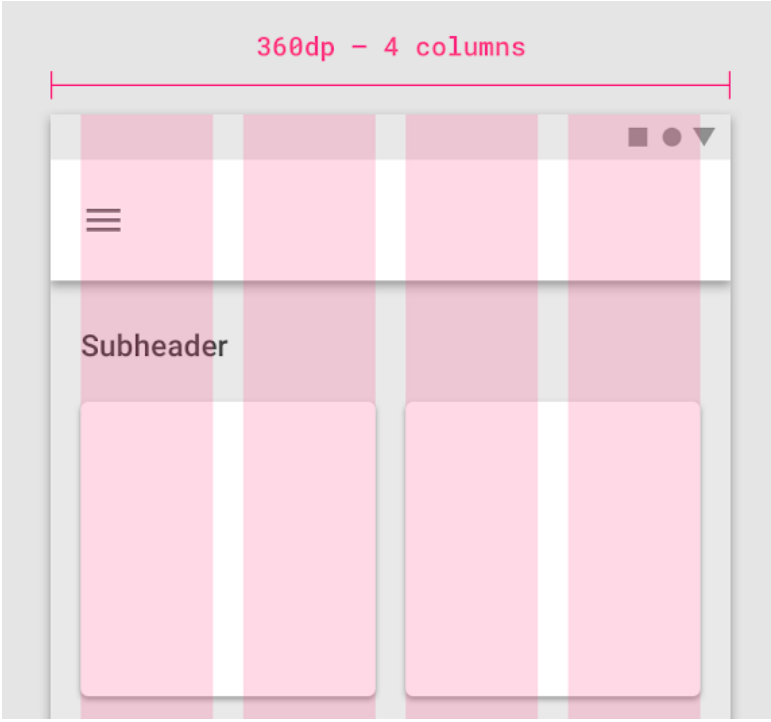
Sistema de 12 colunas

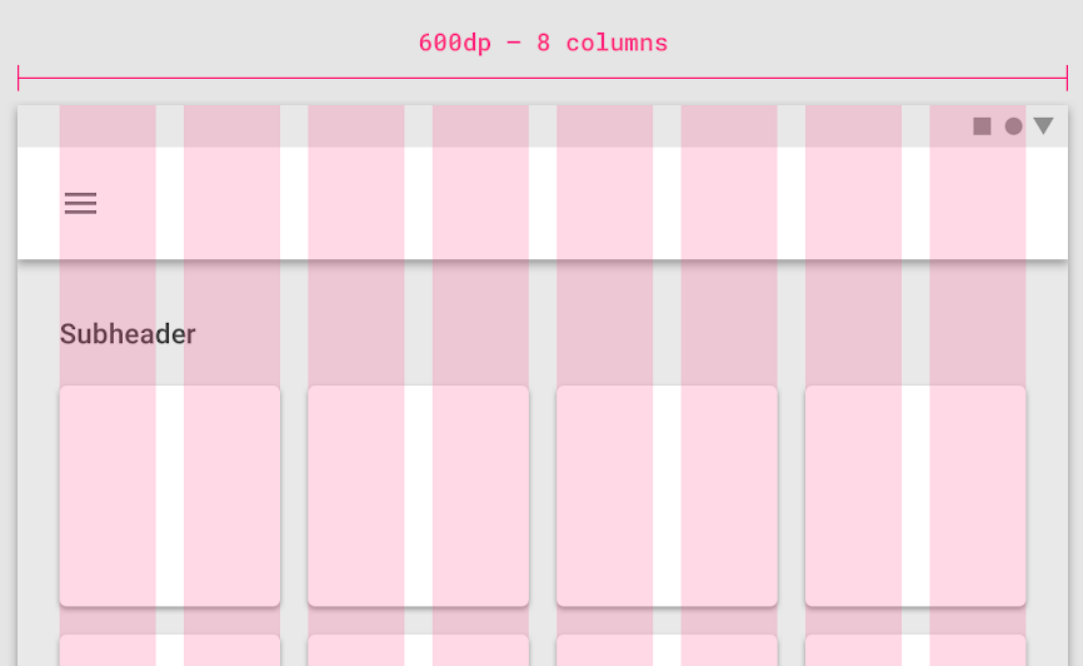
- Um modelo de layout que divide a tela em 12 colunas iguais;
- Usado para organizar o conteúdo de forma proporcional e alinhada, especialmente para layouts responsivos.
- Divide o espaço horizontal disponível em 12 colunas de igual largura, permitindo o posicionamento e dimensionamento precisos dos elementos dentro do layout.

Vantagens

- Tornou-se uma escolha amplamente utilizada devido:
  - Flexibilidade: colunas podem ser combinadas ou redimensionadas;
  - Responsividade: adaptam-se a diferentes tamanhos de tela;
  - Eficiência: estrutura predefinida.


1
2
Subheader
3


360dp – 4 columns
Subheader


600dp – 8 columns
Subheader

Exemplo

```
.container {
display: grid;
grid-template-columns:
repeat(12, 1fr);
gap: 10px;
padding: 10px;
}

.header {
grid-column: span 12;
background: #4CAF50;
padding: 20px;
}

.
```

```
.sidebar {
grid-column: span 3;
background: #2196F3;
padding: 20px;
}

.content {
grid-column: span 9;
background: #FFC107;
padding: 20px;
}

.footer {
grid-column: span 12;
background: #f44336;
padding: 20px;
}
```

RESPONSIVIDADE

Responsividade

- Capacidade do layout se adaptar a diferentes tamanhos de tela.

- O mesmo site precisa funcionar bem em qualquer tela.

Media Queries

- Permitem aplicar CSS dependendo do tamanho da tela.

```
@media (condição) {
        /* regras CSS */
}
```

Layout

- Ao redimensionar um layout para diferentes tamanhos, o grid responsiva ajusta as margens e as larguras do corpo, bem como o número de colunas.

Screen size	Margin	Body	Layout columns
Extra-small (phone)			
0-599dp	16dp	Scaling	4
Small (tablet)			
600-904	32dp	Scaling	8
905-1239	Scaling	840dp	12
Medium (laptop)			
1240-1439	200dp	Scaling	12
Large (desktop)			
1440+	Scaling	1040	12

Seletor de atributo

[class^="col-"]: começa com

[class$="col"]: termina com

[class*="col"]: contém

FUNÇÕES IMPORTANTES

minmax()

- Define um intervalo de tamanho.

grid-template-columns: minmax(200px, 1fr);

mínimo: 200px

máximo: ocupa espaço disponível

repeat()

- Repete colunas e linhas automaticamente
- Evita escrever código repetido

```
grid-template-columns: repeat(3, 1fr);
```

auto-fit e auto-fill

- Criam colunas automaticamente conforme o espaço disponível.
- auto-fit: ajusta e ocupa espaço.
- auto-fill: mantém espaços vazios.

grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

fit-content

- Define o tamanho baseado no conteúdo.
- Limita o crescimento sem quebrar o layout.
- grid-template-columns: fit-content(300px);
  - Cresce até 300px;
  - Mas respeita o conteúdo.